

Assignment-01

Basics of Verification

1. Initialize an array using:
 - Default values
 - Repetitive values
 - Unique values
2. Create a fixed-size array and display its contents.
3. Create a dynamic array, allocate memory dynamically, and display the elements.
4. Create a queue and perform push and pop operations.
5. Create a class consisting of three unsigned integer data members initialized to 45, 78, and 90. Display the values using a class method.
6. Create a function that multiplies two unsigned integers and verifies the result against an expected value. Display "TEST PASSED" or "TEST FAILED".
7. Create a task that generates stimulus for:
 - addr (6-bit)
 - wr (1-bit)
 - en (1-bit)

Apply stimulus on the positive edge of a 25 MHz clock.
8. Create a function that generates and returns 32 multiples of 8 (0, 8, 16, ..., 248). Store them in a local array and display all values.
9. Create a class with three 8-bit variables a, b, and c. Implement a custom constructor to initialize these variables. Verify using values 2, 4, and 56.
10. Create a class containing three 4-bit variables a, b, and c. Implement a task that:
 - Calculates their sum
 - Returns the result
 - Displays all inputs and output
11. Create a deep copy method for a Generator class and verify that the copied object contains identical values but occupies a different memory location.

Randomization and Constraints

12. Create a class with 8-bit addr and data fields. Constrain:

- addr between 16 and 63
- data to even values only

Generate and display 20 randomized transactions.

13. Create a class with:

- rand addr
- rand data

Apply inline constraints during randomization to generate:

- Addresses between 100 and 150
- Data values greater than 200

14. Create a transaction class and implement:

- pre_randomize()
- post_randomize()

Display messages before and after randomization